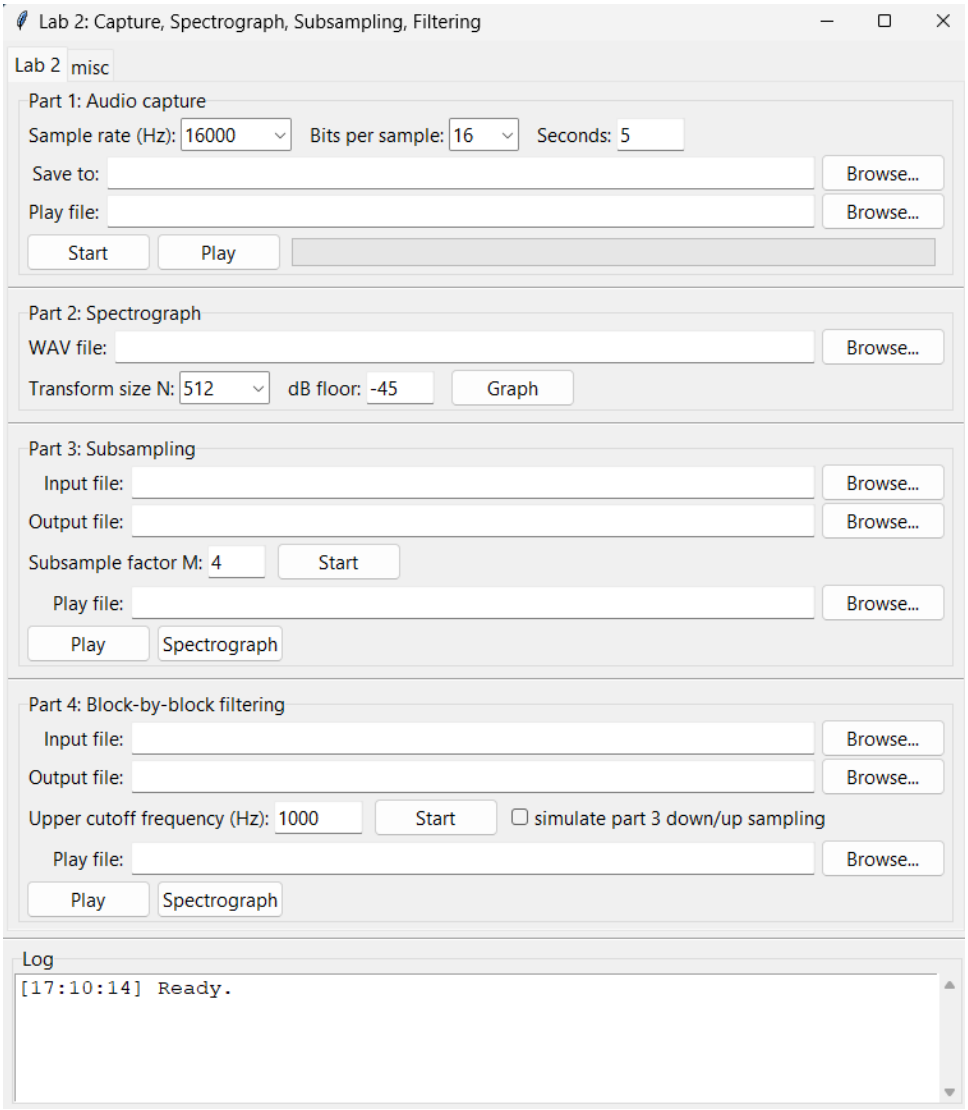
Lab 2

In this lab we will investigate aliasing with audio files. You will first develop an app with Claude, and then use it to investigate aliasing. Your app does NOT have to look exactly like this, there is a lot of flexibility, but this will give you a rough idea of what I'm after:



I am only going to spell out the requirements in rough detail, because part of what I want you to do is write a solid requirements document. This will be more and more

important in the future as your careers develop, as coding is taken over by agents.

The app basically needs to do the following. The UI can be substantially different than the above, but you need this basic functionality:

1. Write the code in python using tkinter for the UI
2. You need to be able to capture an audio file of an arbitrary length from your PC's microphone. You need to be able to specify the sample rate (have at least 8kHz, 16kHz, and 48kHz as options). You need to support at least 16 bits per sample for now, and use a signed integer format. (Note that for 8 bits per sample you may be forced to use an unsigned byte format). Save your files in .wav uncompressed format
3. You should be able to playback a file that you have captured
4. You should be able to compute a spectrograph and graph it (see image at the end of this document) Compute the power spectrum on a block-by-block basis (make the block size a variable) and use the transform I defined in class
    i. Use dB on the y-axis relative to the highest power
    ii. Fade from blue to red where blue is low power relative to the max power, and red is the max power (0dB). Have a noise floor and clip all values below the noise floor. I find -45dB to be good (Pure blue -45dB, pure red 0dB)

iii. Don't forget to appropriately include both the sine and cosine coefficients that are at the same frequency when computing the power!

5.) You need to be able to sub-sample a selected file by simply taking every M'th sample. DO NOT filter before taking every M'th sample. The whole goal is to investigate the effect of aliasing. You need to be able to write the sub-sampled audio to a .WAV file, play it back, and generate a spectrograph

   a. Once you have sub-sampled WITHOUT filtering, you can tell Claude that you want to upsample it by the same factor that you downsampled with. Tell Claude to use "standard high-quality multi-rate DSP filters and techniques" for the upsampling. This will allow the file to be at the same sample rate as the original file, and make spectrograph comparisons easy.

6.) Note that the k'th coefficient of 0, 1, 2, …, N/2 where N is the transform size corresponds to frequency (k/N)*Fs where Fs is the sampling frequency. Make your plots show the actual frequency based on the sampling rate

7.) Finally, you need to be able to do block-by-block filtering using a chosen block size. You should specify a desired 'lowpass cutoff frequency': then forward transform a block, throw away all frequencies higher than that (set those transform coefficients to 0). Make sure to retain BOTH the sine and cosine components that are below the cutoff! Repeate this for every block

a. Instruct Claude to use exactly the orthonormal transform we developed in class! However, tell it to use an efficient FFT based implementation. We won't cover how the FFT works in this class, but it is just a fast algorithm for doing the matrix multiply that exploits the fact that the basis functions are sines/cosines

Once you have your app up and running you are ready to do some investigations with it. You may find it useful to add a second tab to your app, and have claude generate some fixed tone .wav files for you. Put your results together in a single pdf file and be sure to label your results! I'm not looking for a lab writeup, but I want something like this before each result:

"below is the graph that shows….."

And if I ask you to discuss or explain something put that in your paper too. Keep your results in the order in which I ask for them below

Do these things:
a.) Generate pure tones at 16 kHz sampling with 16 bits per sample that last for 5 seconds. Generate them at the frequencies: {500, 1000, 2500, 3000, 5000, 7000}

b.) Run the 2500 kHz tone through your spectrograph. Include your spectrograph in your submission. Explain what you see in a couple of sentences

c.) Run the 5000 kHz tone through your spectrograph, include in your submission, and explain what you see

d.) Subsample the 5000 kHz tone by a factor of 2. Run it through the spectrograph, and include in your submission. Is this what you expect? Explain what you see. Listen to the files, do they sound the same?

e.) capture 10 seconds of a music clip that is consistently playing and ideally has both bass and treble components in it. Generate a spectrograph. Include your music clip and the spectrograph in your submission. Can you correlate the spectrograph with the music when you look/listen to them at the same time?

f.) subsample your music by 2, 4, and 5: describe what you hear? Look at the spectrographs. What do you see? Any observations?

g.) Now use your block-by-block filtering interface to filter your music file before subsampling: filter out all frequencies above 4 kHz when subsampling by 2; everything above 2 kHz when subsampling by 4; and everything above 1.6 kHz when sub-sampling by 5. Use a block size of 16,284 for doing this filtering. Listen to the files. Do they sound the same as your part f results? Which do you prefer?